Idea in your project	Comes from	Is it from these papers?
KG built from **hyperlinked XML**	**Your own data**	❌ No — both papers work on flat prose; AutoKG *avoids* hyperlinks
Typed edges from section headings	**Your XML structure**	❌ No — neither paper has structure
LLM triple extraction + generation	Modern Graph RAG (2024–26)	❌ No — both papers are pre-LLM
Internal alignment (bare-text mentions → nodes)	✅ **AutoKG**	✅ Yes — 1 of 2 keepers
Multi-hop traversal for retrieval	✅ **AutoKG**	✅ Yes — 2 of 2 keepers
Coreference resolution	✅ **Textbook-to-Triples** (concept only)	✅ Yes — but your LLM does it free

Phase 1: Input processing

Filter English → segment into chunks → batch

|

▼

Phase 2: Triple extraction (3 LLM passes) ← the edges get made here

Entity–Entity (P_EE)

Entity–Event (P_EV)

Event–Event (P_VV)

→ JSON triples

|

▼

Phase 3: Schema induction ← what makes it a KG, not just triples

LLM generates abstract concept phrases for every

entity / event / relation → concept nodes (C)

|

▼

Phase 4: Graph construction

Create entity + event + concept nodes

Add relation edges

Link every node to its concept nodes

→ finished graph G = (V, E, C, ϕ, ψ) → ATLAS

The three retrievers in the paper

1. Think-on-Graph (ToG) — *pure graph traversal*
Walks the graph step by step: extract entities from the query → find starting nodes → expand paths one hop at a time → prune to the top-N most relevant paths (an LLM scores them 1–5) → check if enough to answer → generate. This is their **pure-graph, no-hybrid** retriever — the one that matches your v1. (Full pseudocode is in their Algorithm 1.)

2. HippoRAG — *Personalized PageRank*
Uses Named Entity Recognition to pick seed nodes from the query, then runs **PageRank** over the graph from those seeds to score which nodes/passages matter. Neurobiologically-inspired ("memory graph").

3. HippoRAG2 — *improved PageRank* ← **their best performer**
An upgraded HippoRAG: it embeds query and edges, selects top edges, has an **LLM filter** them, then runs **Personalized PageRank** with a damping factor (they used 0.9). In their results tables, **HippoRAG2 gave the highest scores** of the three.

Quick comparison

Retriever	Method	Hybrid?	Notes
ToG	Step-by-step graph traversal	No (pure graph)	Simplest; matches your v1; scored lowest of the three
HippoRAG	Personalized PageRank from NER seeds	No (pure graph)	Middle
HippoRAG2	PageRank + edge embeddings + LLM edge-filter	Mostly graph	**Best results in the paper**

HippoRAG2 is the best retriever for your use case — but with an important "start simpler" qualifier. Let me separate "best" from "best to start with," because they're different and both matter for you.

Best overall: HippoRAG2

For *your* specific setup, HippoRAG2 (Personalized PageRank) is the strongest fit, for reasons that go beyond just "it won the paper's benchmarks":

- **It matches how your graph is shaped.** You have a *dense* structure — hyperlinks + entity edges + causal edges + concept nodes all connecting things. PageRank thrives on exactly that kind of connectivity; it spreads importance across your links to find relevant nodes even when they're a few hops from the query. Plain traversal (ToG) doesn't exploit density as well.
- **Your hyperlinks become ranking signal, not just paths.** In PageRank, a heavily-linked node naturally scores higher. Your hyperlink graph *is* that signal — HippoRAG2 uses it automatically. ToG just walks edges; it doesn't reward well-connected nodes.
- **It handles multi-hop better** — which is where your value is (cross-document questions, causal chains like wear→error state). It was the paper's top performer precisely on multi-hop datasets.
- **It's still non-hybrid** — fits your "no hybrid" rule. (It uses embeddings to pick seed edges, but that's graph entry, not a parallel vector retrieval.)

The one-sentence reason

You use HippoRAG2 because your graph's defining feature is **dense connectivity** (hyperlinks + entity + causal + concept edges), and PageRank is the one method that turns that connectivity into retrieval *strength* — while GraphRAG/LightRAG/MiniRAG target the wrong question type or trade away accuracy, and RAPTOR/ToG don't exploit your graph structure at all.

And you *start* with ToG only because it's the simplest way to get the pipeline running and confirm your graph is good — then you move to HippoRAG2, which is built for exactly the kind of graph you're creating.

So the choice isn't arbitrary: each alternative was tested and loses on a dimension that matters specifically to a hyperlink-dense, factual, multi-hop engineering use case — which is precisely what you have.

	On Cosmos DB
v1 retriever	**ToG** — neighbor queries are native Gremlin, minimal friction ✅
v2 retriever	**HippoRAG2** — works, but pull subgraph via Gremlin → PageRank in NetworkX
What changes vs Neo4j	Only the query language (Gremlin instead of Cypher). Retriever choice is the same.

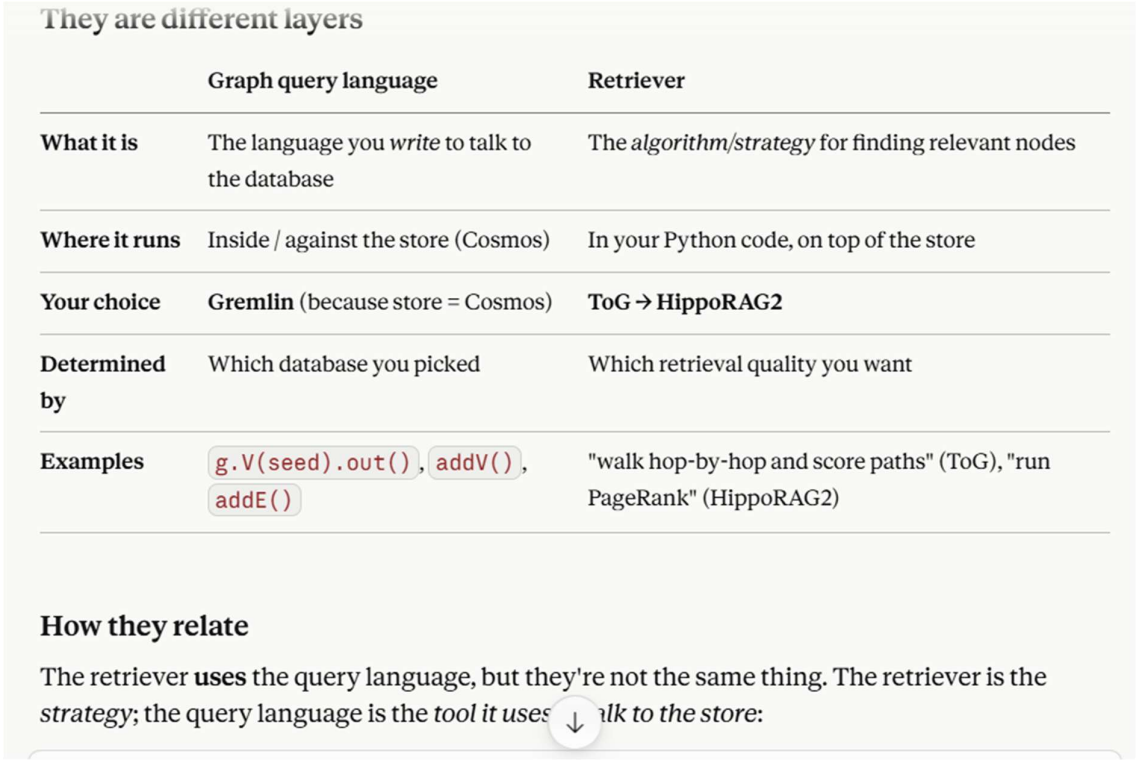
ToG (Think-on-Graph) for v1, then HippoRAG2 for v2. That's your retriever — unchanged by any of the Cosmos/NetworkX/query-language discussion, because the retriever choice is independent of all of that.

Here it is plainly:

	Retriever	When	Why
v1	**ToG** (pure graph traversal)	build now	Simplest, matches your no-hybrid scope, only needs "get neighbors" queries (native to Gremlin/Cosmos), easy to debug
v2	**HippoRAG2** (Personalized PageRank)	after measuring v1	The paper's best performer, best fit for your dense hyperlinked graph, upgrade if multi-hop underperforms

What ToG does (v1): extract entities from the question → find seed nodes → walk the graph one hop at a time (`g.V(seed).out()`) → LLM scores the paths → stop when it has enough → generate the answer.

What HippoRAG2 does (v2): embed query + ↓ es → pick top edges → LLM filters them → run Personalized PageRank over the graph (in NetworkX) → return top nodes → generate.



They are different layers

	Graph query language	Retriever
What it is	The language you *write* to talk to the database	The *algorithm/strategy* for finding relevant nodes
Where it runs	Inside / against the store (Cosmos)	In your Python code, on top of the store
Your choice	**Gremlin** (because store = Cosmos)	**ToG → HippoRAG2**
Determined by	Which database you picked	Which retrieval quality you want
Examples	`g.V(seed).out()`, `addV()`, `addE()`	"walk hop-by-hop and score paths" (ToG), "run PageRank" (HippoRAG2)

How they relate

The retriever **uses** the query language, but they're not the same thing. The retriever is the *strategy*; the query language is the *tool it uses to talk to the store*:

This is actually the **most important design decision** in your project. Let's analyze it from a research perspective instead of simply saying "yes" or "no."

First, what is an ontology?

An ontology answers three questions:

1. **What types of things exist?** (Classes)
2. **How are they related?** (Relationships)
3. **What rules do they follow?** (Constraints/Reasoning)

For example, in an aircraft domain:

Classes:

Aircraft

Engine

Gearbox

Sensor

FailureMode

Requirement

Relationships:

---

contains

installed_in

causes

depends_on

This is useful when you want to reason over these domain concepts.

Now let's look at your project

Your data looks like this:

Power_Gearbox.xml

|

contains hyperlink

▼

Robust_Design.xml

|

contains hyperlink

▼

Boundary_Diagram.xml

Every XML file is already a **complete engineering document**.

Your graph is:

Power_Gearbox.xml

|

explains

▼

Robust_Design.xml

Notice something important:

Your nodes are documents, not engineering entities.

What is your research question?

Your project is trying to answer:

"Given a user query, which XML documents should I retrieve?"

NOT

"Which failure mode belongs to which gearbox?"

NOT

"Which sensor is installed in which engine?"

NOT

"What is the parent class of a gearbox?"

These are ontology-driven questions.

Your question is **document retrieval**.

What information do you actually need?

Suppose the user asks:

Explain Robust Design.

Your system needs to find:

Power_Gearbox.xml

↓

Robust_Design.xml

↓

System_Engineering.xml

It doesn't need to know:

Robust Design

is_a

DesignMethod

That classification doesn't help retrieve the document.

What does HippoRAG2 need?

HippoRAG2 works on a graph.

It needs:

Node

↓

Edge

↓

Connected Graph

Example:

A

|

explains

▼

B

|

depends_on

▼

C

HippoRAG2 traverses this graph.

It does **not** require:

Class

↓

Subclass

↓

OWL reasoning

Imagine adding an ontology

You would first create:

Document

EngineeringArtifact

DesignMethod

Requirement

System

Subsystem

FailureMode

Component

Procedure

Now every XML must be classified.

For 20,000 XMLs:

Power_Gearbox.xml

↓

EngineeringArtifact?

or

Procedure?

or

Requirement?

or

DesignMethod?

That becomes another large project.

Will retrieval improve?

Suppose your graph is

Power_Gearbox.xml

↓

explains

↓

Robust_Design.xml

Adding

Robust_Design

is_a

DesignMethod

doesn't automatically improve retrieval.

If the user asks:

Explain Robust Design.

The graph already reaches the correct document.

When is ontology useful?

Ontology becomes valuable when you want reasoning like:

Engine

contains

Gearbox

Gearbox

contains

Bearing

Then you can ask:

Which engines contain bearings?

The system can infer:

Engine

↓

Gearbox

↓

Bearing

That is ontology-based reasoning.

Your project is different

Your project is:

User Question

↓

Find relevant XML documents

↓

Read XML documents

↓

Answer

You are retrieving **documents**, not inferring engineering facts.

What should your graph contain?

Just this:

Power_Gearbox.xml

|

explains

▼

Robust_Design.xml

Robust_Design.xml

|

depends_on

▼

Boundary_Diagram.xml

Boundary_Diagram.xml

|

belongs_to

▼

Systems_Engineering.xml

That is sufficient for GraphRAG.

Could you add an ontology later?

Yes.

After building the graph, you might notice patterns:

Document

    |

    ├— Design Document

    ├— Requirement

    ├— Procedure

    ├— Failure Report

Then the ontology becomes an **enhancement**, not a prerequisite.

My conclusion for your research

For your **Rolls-Royce XML GraphRAG POC**, I recommend **not starting with an ontology** because:

1. **Your retrieval unit is the XML document**, not individual engineering entities.
2. **The hyperlinks and LLM-generated predicates already provide the graph structure** needed for GraphRAG.
3. **HippoRAG2 (or any graph retriever) requires a connected graph, not a formal ontology.**
4. **Designing an ontology for ~20,000 engineering XML documents would require significant manual or semi-automatic classification, without a clear benefit to your initial retrieval task.**

If, later in the project, you want to support richer semantic queries (for example, "show all design methods related to gearbox failures"), then introducing a lightweight ontology would make sense. For the current goal—**retrieving the right XML documents and generating answers**—the document graph itself is the essential component.

For **your GraphRAG POC**, you should not say *"ontology is not useful."* That would be incorrect.

Instead, you should say:

"A formal ontology is not required for the initial phase of our GraphRAG because our objective is document retrieval rather than domain reasoning."

Here is a proper explanation you can use in your report, presentation, or research discussion.

Why We Are Not Using an Ontology in Our GraphRAG POC

1. Our graph is document-centric, not entity-centric

Our GraphRAG is built around **XML documents**.

- **Node = XML Document**
- **Edge = Semantic relationship between XML documents**

Example:

Power_Gearbox.xml

|

explains

▼

Robust_Design.xml

Since each node already represents a complete document, there is no immediate need to classify it into multiple ontology classes.

---

2. Our objective is document retrieval

The goal of our system is:

User Query

|

▼

Retrieve the most relevant XML documents

|

▼

Generate the answer using the retrieved documents

We are **not** trying to answer questions such as:

- Which component is a subtype of another?
- Which failure mode belongs to which subsystem?
- What is the hierarchy of engineering concepts?

Those types of questions require an ontology.

Our system only needs to retrieve relevant documents.

---

3. Hyperlinks already define the graph structure

The XML files already contain hyperlinks such as:

doc:Robust_Design

doc:Boundary_Diagram

doc:System_Engineering

These hyperlinks naturally create connections between documents.

The LLM then converts these references into semantic relationships:

Power_Gearbox.xml

|

explains

▼

Robust_Design.xml

Therefore, the graph structure is derived directly from the documents without requiring a predefined ontology.

---

4. The graph retriever does not require an ontology

Graph retrieval methods (such as HippoRAG2 or graph traversal algorithms) operate on:

- Nodes
- Edges
- Connectivity

They do not require:

- Class hierarchies

- OWL reasoning
- RDFS reasoning
- Complex ontology definitions

As long as the graph is connected through meaningful relationships, retrieval can be performed effectively.

---

5. Creating an ontology would increase project complexity

Our dataset contains approximately **20,000 XML documents**.

Building a formal ontology would require:

- Identifying domain classes
- Defining subclasses
- Classifying every XML document
- Maintaining ontology consistency

This adds significant effort without directly improving the primary objective of our POC, which is document retrieval.

---

6. We follow a data-driven approach

Instead of designing a complete ontology before analyzing the data, we first:

1. Build the knowledge graph.
2. Evaluate GraphRAG retrieval performance.
3. Analyze the graph.
4. Introduce an ontology only if recurring domain concepts justify it.

This iterative approach is more practical for a Proof of Concept.

When would an ontology become useful?

An ontology becomes valuable if future requirements include:

- Semantic reasoning
- Domain inference
- Class hierarchies

- Constraint validation
- SPARQL queries over domain concepts

For example:

Component

|

contains

▼

Gearbox

|

contains

▼

Bearing

This enables reasoning such as:

"Which components indirectly contain bearings?"

Such reasoning is outside the scope of our current GraphRAG POC.

Final Justification (Research Style)

A formal ontology is not included in the initial GraphRAG Proof of Concept because the system is designed for document-centric retrieval rather than ontology-driven reasoning. Each XML document is represented as a graph node, and semantic relationships between documents are extracted using hyperlinks and an LLM. This graph structure provides sufficient information for GraphRAG retrieval without the additional complexity of designing and maintaining a domain ontology. If future requirements involve semantic reasoning, hierarchical classification, or rule-based inference, a lightweight ontology can be incorporated as an extension of the existing knowledge graph.

This justification is technically sound, aligns with current GraphRAG practices, and matches the iterative, data-driven philosophy discussed in the article you shared.

Step 1 — Node types (from what appears in the doc)

Reading your P-diagram file, these are the *kinds of things* that show up:

Node type	What it is	From your doc
Document	the page itself	the P-diagram page
SystemConcept	engineering method/artefact	P-diagram, systems-engineering artefact, boundary diagram
System	the thing under study	SOI, Power gearbox
Factor	an influencing factor	noise factor, control factor, HMI, input signals
FailureMode	undesirable outcome	error state, failure mode
Method	a discipline/technique	Robust Design
Event	a happening (for causal edges)	"component wear occurs", "error state emerges"
Concept	abstract bridge type	"disturbance" (from schema induction)

That's **8 node types** — right in the small "10–12" range the articles recommend. From *one* file. You won't need many more.

Step 2 — Relation types (edge vocabulary)

These come straight from your section headings + the causal/definition sentences:

Relation	Where it came from	Example edge
IS_A	"a p-diagram **is a** systems-engineering artefact"	(P-diagram)→(SE artefact)
IDENTIFIES_FACTORS_OF	"identifies factors that influence a SOI"	(P-diagram)→(SOI)
HAS_INPUT_SIGNAL	section "Input signals"	(P-diagram)→(HMI)
HAS_CONTROL_FACTOR	section "Control factors"	(P-diagram)→(control factor)
HAS_NOISE_FACTOR	section "Noise factors"	(P-diagram)→(noise factor)
HAS_ERROR_STATE	section "Error states"	(P-diagram)→(failure mode)
EXPLORES	"explore the available design space"	(control factor)→(design space)

Relation	Where it came from	Example edge
PRECEDES	"boundary diagram created **before** the P-diagram"	(boundary diagram)→(P-diagram)
CAUSES	"component wear **causes** error states"	(wear event)→(error state event)
MITIGATES	"Robust Design... reduce output variation"	(Robust Design)→(noise factor)
SEE_ALSO	section "See also"	(P-diagram)→(related node)
REFERENCES	fallback for untyped links	(P-diagram)→(node)

That's **12 relation types** — again, small on purpose.

Step 3 — Node properties (metadata, not edges)

From the "Export control marking" section — these become **properties on the node**, not relationships:

export_jurisdiction, export_rating, export_date, teamcenter_url, reference_id, title

Your complete PoC ontology (from one file)

NODE TYPES (8):

Document · SystemConcept · System · Factor

FailureMode · Method · Event · Concept

RELATION TYPES (12):

IS_A · IDENTIFIES_FACTORS_OF · HAS_INPUT_SIGNAL

HAS_CONTROL_FACTOR · HAS_NOISE_FACTOR · HAS_ERROR_STATE

EXPLORES · PRECEDES · CAUSES · MITIGATES

SEE_ALSO · REFERENCES

NODE PROPERTIES:

title · document_type · export_jurisdiction · export_rating

export_date · teamcenter_url · reference_id · source_xml_location

That's your entire ontology. 8 node types, 12 relations, a handful of properties. That's it — small, queryable, and built entirely from what's already in your document.

The key things to notice

1. **You didn't design this — you read it off the document.** Every type came from a section heading, a hyperlink, or a sentence. This is exactly why a formal ontology exercise is unnecessary for you: your XML structure *is* the ontology.
2. **This is the "start small, evolve" model in action.** This ontology fits the P-diagram. When you process the SAR report, you'll hit a *collision* — it needs Threat (node) and RELATES_TO_THREAT (relation) that aren't here yet. So you **add those two** and move on. That's the data-exploration loop both articles describe: don't design upfront, extend when a real document demands it.
3. **How you'd use it:** freeze this list, put the relation types in your LLM extraction prompt as "choose the relation only from this set," and use the node types as the labels. That single constraint keeps your graph consistent and traversable.

What to do next

For your PoC, this one-file ontology is your **starting base**. As you feed in more document types, it grows by a type or two each time — never a redesign. After ~5–10 documents of each type, it'll stabilize and you'll rarely add anything.

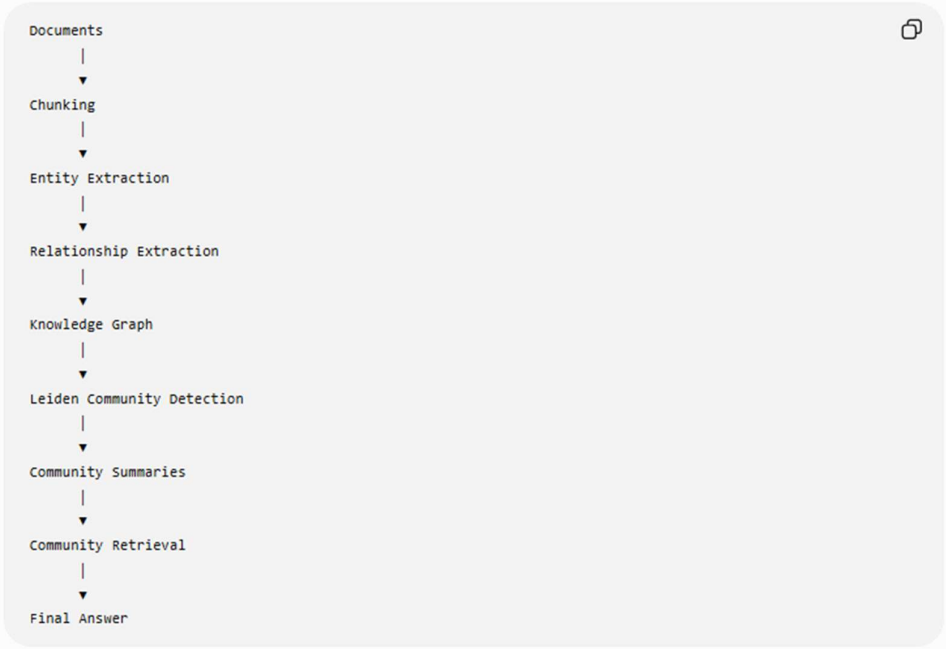
Want me to write this up as the **actual extraction-prompt constraint block** — the exact text you paste into your LLM prompt that says "extract nodes as one of these 8 types, relations as one of these 12" — so it's ready to drop into your pipeline? That turns this ontology from a list into the working guardrail your extractor uses.

Our current use case does not justify building a formal ontology because our GraphRAG is document-centric rather than entity-centric.

Your Use Case

Instead of extracting **entities**, you extract **document relationships**.

2. Microsoft GraphRAG



Architecture Comparison

1. AutoSchema